Code Verification in Rust with Verus

Matt Grimes

Oct 30 2026

What is code verification

Z3

Verus

Verification as LLM Guardrails

What does it mean for code to be "correct?"

To ask if code is "correct," we have to formulate a formal claim about what the code does.

Taking a max

If *max* returns the max of two integers, x and y, we expect the following properties:

- $max(x, y) \in \{x, y\}$
- $max(x, y) \geq x$
- $max(x, y) \geq y$

Taking the max of two integers

We can state the logical claim that the implementation is correct as

$$\forall x, y, m \in \mathbb{Z},\ m = \max(x, y) \implies (m = x \vee m = y) \wedge m \geq x \wedge m \geq y$$

In general, $a \implies b$ is equivalent to $\neg a \vee b$. We can thus rewrite

$$\forall x, y, m \in \mathbb{Z},\ \neg(m = \max(x, y)) \vee ((m = x \vee m = y) \wedge m \geq x \wedge m \geq y)$$

Equivalently, the implementation of max has a bug if and only if

$$\exists x, y, m \in \mathbb{Z},\ (m = \max(x, y)) \wedge \neg((m = x \vee m = y) \wedge m \geq x \wedge m \geq y)$$

The general formulation

Let s and s' denote program states, and let $C(s, s')$ mean that program C can terminate in s' when started in s.

$$\forall s, s',\ P(s) \wedge C(s, s') \Longrightarrow Q(s') \quad := \quad \{P\}\ C\ \{Q\}$$

- P: a predicate on the initial state (the precondition).
- Q: a predicate on the final state (the postcondition).
- Initial values can be retained in the state to express input–output relations.

Applied to max

In our example we have

- P: x and y are integers
- C: m is given by the implementation of max
- Q: $m \geq x \wedge m \geq y \wedge (m = y \vee m = x)$

What about the implementation?

In pseudo-code, we can define a naive implementation of max:

```
fn max(x: int, y: int) -> (m: int)
{
    if x > y
    {
        return x;
    } else
    {
        return y;
    }

}
```

Logically, this implementation can be expressed by

$$m = \max(x, y) := ((x > y) \wedge (m = x)) \vee (\neg(x > y) \wedge (m = y))$$

Putting it all together

We can combine everything on the prior slides into the following statement: our implementation of max is *incorrect* if and only if $\exists x, y, m \in \mathbb{Z}$ such that

$$\underbrace{[((x > y) \wedge (m = x)) \vee (\neg(x > y) \wedge (m = y))]}_{m \text{ given by } \max(x,y)} \wedge \neg \underbrace{[(m = x \vee m = y) \wedge m \geq x \wedge m \geq y]}_{m \text{ satisfies definition of max}}$$

We can see by inspection that the claim is false, there is no counter-example, and the implementation is proven correct.

Z3: satisfiability relative to a theory

In the last section, we asked whether or not $\exists x, y, m \in \mathbb{Z}$ such that

$$\underbrace{[((x > y) \wedge (m = x)) \vee (\neg(x > y) \wedge (m = y))]}_{m \text{ given by } \max(x,y)} \wedge \neg \underbrace{[(m = x \vee m = y) \wedge m \geq x \wedge m \geq y]}_{m \text{ satisfies definition of max}}$$

This is exactly the type of statement the SMT/SAT solver Z3 is designed to answer.

Z3

SMT: satisfiability modulo theories

Given a formula and a background "theory", is the formula satisfiable?

- Boolean combinations of constraints in supported theories.
- Examples: integer arithmetic, fixed-width bit-vectors, arrays.
- The theory determines the meaning of symbols such as $+$ and $\leq$.

For our example: linear constraints over the integers.

Source: Microsoft's Z3 Guide

Prove a property by searching for its violation

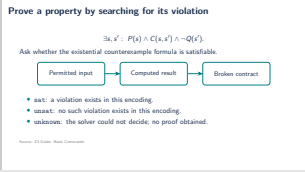
$$\exists s, s' : \; P(s) \wedge C(s, s') \wedge \neg Q(s').$$

Ask whether the existential counterexample formula is satisfiable.



- `sat`: a violation exists in this encoding.
- `unsat`: no such violation exists in this encoding.
- `unknown`: the solver could not decide; no proof obtained.

Source: Z3 Guide: Basic Commands



A timeout also leaves the obligation unproved. With abstractions, a satisfying model may require interpretation before calling it a real program counterexample. Our small exact arithmetic example has a direct interpretation.

Ask Z3 to find any bugs

```
(define-fun is_max ((m Int) (x Int) (y Int)) Bool
    (and
        (>= m x)
        (>= m y)
        (or
            (= m x)
            (= m y)
        )
    )
)
(define-fun max ((x Int) (y Int)) Int
    (ite (> x y) x y)
)

(declare-const x Int)
(declare-const y Int)
(declare-const m Int)
(assert
    (and
        (= m (max x y))
        (not (is_max m x y))
    )
)
(check-sat)
(get-model)
```

Result: `unsat`

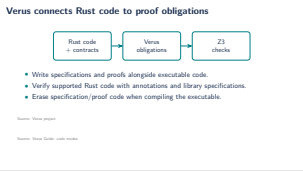
Verus connects Rust code to proof obligations



- Write specifications and proofs alongside executable code.
- Verify supported Rust code with annotations and library specifications.
- Erase specification/proof code when compiling the executable.

Source: Verus project

Source: Verus Guide: code modes



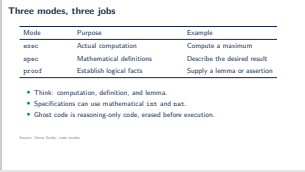
Verus supports a subset of Rust; existing crates are not automatically fully verified. The diagram is conceptual, not a complete description of the internal compiler pipeline.

Three modes, three jobs

Mode	Purpose	Example
`exec`	Actual computation	Compute a maximum
`spec`	Mathematical definitions	Describe the desired result
`proof`	Establish logical facts	Supply a lemma or assertion

- Think: computation, definition, and lemma.
- Specifications can use mathematical `int` and `nat`.
- Ghost code is reasoning-only code, erased before execution.

Source: Verus Guide: code modes



Machine integers still matter in exec code. Mathematical types let the specification describe arithmetic without silently wrapping. Proof functions can package reusable arguments.

Implementing max in Rust

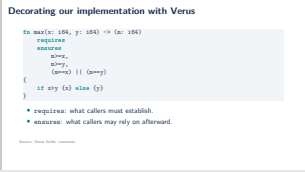
```
fn max(x: i64, y: i64) -> i64
{
    if x>y {x} else {y}
}
```

Decorating our implementation with Verus

```
fn max(x: i64, y: i64) -> (m: i64)
    requires
    ensures
        m>=x,
        m>=y,
        (m==x) || (m==y)
{
    if x>y {x} else {y}
}
```

- `requires`: what callers must establish.
- `ensures`: what callers may rely on afterward.

Source: Verus Guide: contracts



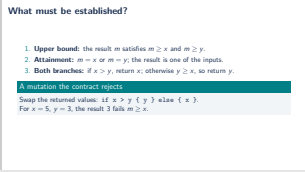
The complete file is examples/max.rs. There are no preconditions on the two i64 inputs. The executable function only compares and selects, so there is no arithmetic-overflow obligation for its body. The three postconditions characterize the maximum.

What must be established?

1. **Upper bound:** the result m satisfies $m \geq x$ and $m \geq y$.
2. **Attainment:** $m = x$ or $m = y$; the result is one of the inputs.
3. **Both branches:** if $x > y$, return x; otherwise $y \geq x$, so return y.

A mutation the contract rejects

Swap the returned values: `if x > y { y } else { x }`.
For $x = 5$, $y = 3$, the result 3 fails $m \geq x$.



No addition occurs in this implementation. The proof is a case split in a total order. At equality either input is a valid maximum, so changing $>$ to $>=$ is not a bug; swapping the returned branches is. The attainment clause matters: an arbitrary upper bound need not be the maximum.

Using a verified function is applying a lemma

Let $M(a, b)$ denote the verified function. Its contract gives

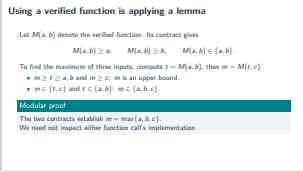
$$M(a, b) \geq a, \qquad M(a, b) \geq b, \qquad M(a, b) \in \{a, b\}.$$

To find the maximum of three inputs, compute $t = M(a, b)$, then $m = M(t, c)$.

- $m \geq t \geq a, b$ and $m \geq c$: m is an upper bound.
- $m \in \{t, c\}$ and $t \in \{a, b\}$: $m \in \{a, b, c\}$.

Modular proof

The two contracts establish $m = \max\{a, b, c\}$.
We need not inspect either function call's implementation.



All inputs are representable machine integers; comparison and selection stay in the same type. This is ordinary lemma application followed by transitivity. Repeating this argument suggests the maximum-over-a-slice algorithm on the next slides.

A loop invariant is an induction hypothesis

For a nonempty slice $a = (a_0, \ldots, a_{n-1})$, after processing i elements:

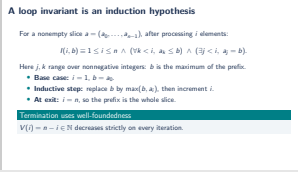
$$I(i, b) \equiv 1 \le i \le n \ \wedge \ (\forall k < i,\ a_k \le b) \ \wedge \ (\exists j < i,\ a_j = b).$$

Here j, k range over nonnegative integers: b is the maximum of the prefix.

- **Base case:** $i = 1$, $b = a_0$.
- **Inductive step:** replace b by $\max(b, a_i)$, then increment i.
- **At exit:** $i = n$, so the prefix is the whole slice.

Termination uses well-foundedness

$V(i) = n - i \in \mathbb{N}$ decreases strictly on every iteration.

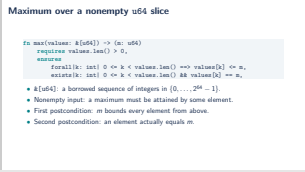


The universal clause is an upper bound; the existential clause gives a witness attaining it. Together they specify the maximum rather than just an upper bound. The index bounds justify reading a_i while the guard holds. Initialization reads a_0, which is why the input must be nonempty.

Maximum over a nonempty u64 slice

```
fn max(values: &[u64]) -> (m: u64)
    requires values.len() > 0,
    ensures
        forall|k: int| 0 <= k < values.len() ==> values[k] <= m,
        exists|k: int| 0 <= k < values.len() && values[k] == m,
```

- `&[u64]`: a borrowed sequence of integers in $\{0, \ldots, 2^{64} - 1\}$.
- Nonempty input: a maximum must be attained by some element.
- First postcondition: m bounds every element from above.
- Second postcondition: an element actually equals m.



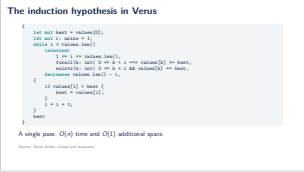
The binary example used i64. For this loop example we use u64 as requested; the same order argument works for both. A slice is a sequence viewed without copying it. The mathematical int indices in the quantifiers are ghost expressions, not runtime iteration.

The induction hypothesis in Verus

```
{
    let mut best = values[0];
    let mut i: usize = 1;
    while i < values.len()
        invariant
            1 <= i <= values.len(),
            forall|k: int| 0 <= k < i ==> values[k] <= best,
            exists|k: int| 0 <= k < i && values[k] == best,
        decreases values.len() - i,
    {
        if values[i] > best {
            best = values[i];
        }
        i = i + 1;
    }
    best
}
```

A single pass: $O(n)$ time and $O(1)$ additional space.

Source: Verus Guide: Loops and invariants



This is the body of max from the preceding slide, read directly from examples/max_iter.rs. best represents b and values represents a. The loop starts at 1 because values[0] has already been processed. While i $<$ len, the next access is in bounds and i $+$ 1 cannot overflow usize. There is no arithmetic on the u64 elements. Verus checks the upper-bound and witness invariants, the postconditions, and the decreasing distance to the end.

What is being trusted?

- The specification captures the intended theorem.
- The translation from code to logical obligations is sound.
- The solver's result and any assumed lemmas are correct.
- Compilation and execution preserve the verified meaning.

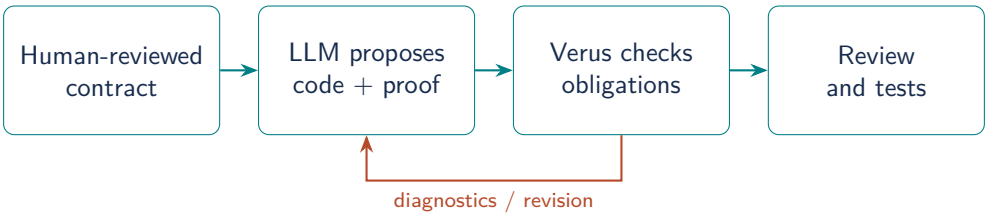
Distinguish proof search from proof checking

Here we rely on the Verus/Z3 verification pipeline.
We are not presenting an independently checked proof certificate.



This is a natural question for an audience familiar with proof assistants. Explain the trust boundary of this workflow without implying that SMT solvers cannot produce proofs or that other proof systems have no trusted components. Our demonstration runs verification; it does not export a certificate to a separate small kernel.

An LLM proposes a construction; verification checks it



Plausibility is not a proof: check the generated construction against a fixed claim.

└An LLM proposes a construction; verification checks it



Plausibility is not a proof: check the generated construction against a fixed claim.

This is a proposed engineering workflow, not a claim that Verus guarantees all LLM output is safe. LLMs can help draft implementations, invariants, and lemmas; the generated proof text itself is not evidence until checked.

A concrete task: count equal pairs

Given a list s, count pairs of positions carrying the same value:

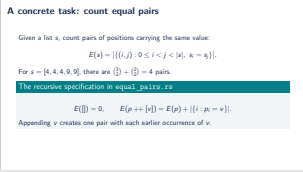
$$E(s) = \left|\{(i,j) : 0 \le i < j < |s|,\ s_i = s_j\}\right|.$$

For $s = [4,4,4,9,9]$, there are $\binom{3}{2} + \binom{2}{2} = 4$ pairs.

The recursive specification in `equal_pairs.rs`

$$E([]) = 0, \qquad E(p \mathbin{+\!\!+} [v]) = E(p) + |\{i : p_i = v\}|.$$

Appending v creates one pair with each earlier occurrence of v.



This counts unordered pairs of distinct positions, each once. It does not count only distinct repeated values or disjoint pairs: three occurrences produce three pairs. The original request was to implement this specification as fast as possible while leaving its definition and the function contract unchanged. The file's filter expression counts matching elements in the prefix.

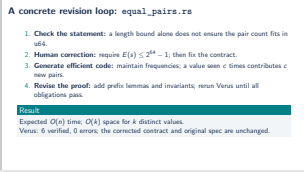
A concrete revision loop: `equal_pairs.rs`

1. **Check the statement:** a length bound alone does not ensure the pair count fits in `u64`.
2. **Human correction:** require $E(s) \leq 2^{64} - 1$; then fix the contract.
3. **Generate efficient code:** maintain frequencies; a value seen c times contributes c new pairs.
4. **Revise the proof:** add prefix lemmas and invariants; rerun Verus until all obligations pass.

Result

Expected $O(n)$ time; $O(k)$ space for k distinct values.
Verus: 6 verified, 0 errors; the corrected contract and original spec are unchanged.

└A concrete revision loop: `equal_pairs.rs`



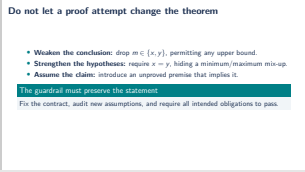
This is the actual interaction behind examples/equal_pairs.rs. An all-equal input has n(n-1)/2 pairs, so the original length precondition was insufficient for a u64 result. The human explicitly corrected the precondition; the model did not silently weaken the theorem to obtain a pass. The implementation uses HashMap::entry to add the old frequency and increment it. Its invariants relate the accumulator to E of the processed prefix and the map to prefix frequencies. Proof revisions included identifying equal filter predicates and selecting quantifier triggers. Prefix monotonicity proves partial sums cannot overflow. Complexity is an algorithmic analysis, not a property proved by this contract; hash-table costs are expected amortized.

Do not let a proof attempt change the theorem

- **Weaken the conclusion:** drop $m \in \{x, y\}$, permitting any upper bound.
- **Strengthen the hypotheses:** require $x = y$, hiding a minimum/maximum mix-up.
- **Assume the claim:** introduce an unproved premise that implies it.

The guardrail must preserve the statement

Fix the contract, audit new assumptions, and require all intended obligations to pass.



These are familiar mathematical ways to prove something other than the original claim. An LLM repair loop can make the same moves. Review hypothesis changes even when they remain consistent. Tool errors and timeouts must not be counted as successful verification. Preserve the command, tool versions, and exact submitted code.

The guardrail has a defined reach

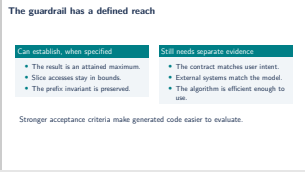
Can establish, when specified

- The result is an attained maximum.
- Slice accesses stay in bounds.
- The prefix invariant is preserved.

Still needs separate evidence

- The contract matches user intent.
- External systems match the model.
- The algorithm is efficient enough to use.

Stronger acceptance criteria make generated code easier to evaluate.

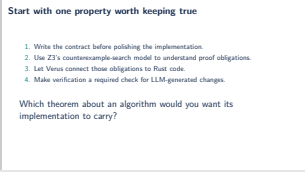


Verification is especially valuable when humans can describe the invariant more reliably than they can inspect every generated implementation. Properties omitted from the contract remain outside the claim.

Start with one property worth keeping true

1. Write the contract before polishing the implementation.
2. Use Z3's counterexample-search model to understand proof obligations.
3. Let Verus connect those obligations to Rust code.
4. Make verification a required check for LLM-generated changes.

Which theorem about an algorithm would you want its implementation to carry?



Invite questions. Possible mathematical examples: a sorting routine preserves the input multiset, Euclid steps preserve the gcd, or a numerical method maintains a certified error bound. Each requires a specification and an appropriate model of arithmetic; floating-point analysis requires additional care.

References and demo files

- Verus Tutorial and Reference

  `https://verus-lang.github.io/verus/guide/`

- Verus source and installation instructions

  `https://github.com/verus-lang/verus`

- Microsoft's Z3 Guide

  `https://microsoft.github.io/z3guide/`

Local examples: `examples/max.rs`, `examples/max_iter.rs`, `examples/max.smt2`, `examples/equal_pairs.rs`.

References and demo files

- Verus Tutorial and Reference
  https://verus-lang.github.io/verus/guide/
- Verus source and installation instructions
  https://github.com/verus-lang/verus
- Microsoft's Z3 Guide
  https://microsoft.github.io/z3guide/

Local examples: `examples/max.rs`, `examples/max_iter.rs`, `examples/max_smt2`, `examples/equal_pairs.rs`.

The README documents build commands and validation status. Source links throughout the deck are clickable. Speaker notes live in the TeX source and are hidden in the audience PDF.